High-Level Synthesis with Game Theoretic Scheduling

Sumon Boiddo
Student ID: 2220054
Hochschule Hamm-Lippstadt
Lippstadt, Germany
sumon.boiddo@stud.hshl.de

***Abstract*—Power consumption has become a critical design dimension in modern VLSI circuits, particularly for battery-powered and embedded systems. This seminar paper presents a game theoretic approach to the scheduling and binding problems of High-Level Synthesis (HLS). In this approach, a Data Flow Graph (DFG) determines which operations are assigned to which control cycles, and each functional unit is modeled as a player in a first-price sealed-bid auction. The Nash equilibrium is then applied to select, for every operation, the functional unit with the lowest power cost, thereby reducing switching activity and overall power consumption. The techniques of functional unit sharing, path balancing, and register assignment are incorporated to further reduce unwanted computations. The approach reduces power without increasing hardware area and with only a small latency cost. Benchmark data drawn from Murugavel and Ranganathan [1] indicate average power improvements of about 6.3% for scheduling, 13.9% for binding, and 11.8% for combined scheduling and binding over linear and integer linear programming baselines. The paper also discusses where this method fits among prior low-power synthesis techniques, its tradeoffs, its applications in IoT and mobile devices, and directions for future work.**

***Index Terms*—High-level synthesis, scheduling, binding, game theory, Nash equilibrium, power optimization, behavioral synthesis.**

I. Introduction

Power consumption in CMOS VLSI circuits has added an important dimension to the design space, in addition to area and timing. The spread of portable and wireless computing has increased the demand for low-power yet high-performance circuits. Higher power consumption shortens battery life in portable systems, increases packaging and cooling costs, and reduces reliability; the reliability of a circuit decreases by roughly 50% for every $10\,^{\circ}\mathrm{C}$ rise in temperature [3]. As a result, minimizing power has become as important as meeting area and timing goals.

High-Level Synthesis (HLS) converts an algorithm-level description, typically written in a language such as C or C++, directly into a hardware design, rather than relying on manual gate-level design [5]. This makes complex hardware feasible to produce. The behavioral synthesis of computational units in HLS proceeds through three main steps: *scheduling*, which decides in which control cycle each operation is executed; *allocation*, which decides how many resources of each type are needed; and *binding*, which decides which specific hardware unit performs each operation [4]. Scheduling and binding strongly influence the power, speed, latency, and hardware utilization of the final circuit, and they waste the most power when handled poorly.

The main focus of this work is to solve the scheduling and binding problem on a Data Flow Graph (DFG) using a game theoretic formulation, such that power consumption is minimized without any increase in area overhead. Although existing methods based on integer linear programming (ILP) and linear programming (LP) can optimize power to some extent, they are often complex and cannot always handle scheduling and binding flexibly, especially when functional unit selection and switching activity must be minimized at the same time [2]. Game theory is well suited to this setting because each hardware unit can be modeled as a player, and the Nash equilibrium provides a stable assignment in which operations are directed to the functional unit with the lowest power cost. The primary goals are therefore to reduce power consumption, to avoid increasing hardware area, and to make scheduling and binding decisions more systematic.

This paper first represents operations and their data dependencies using a DFG and then applies the Nash equilibrium to select a suitable functional unit for each operation. It places this approach within the landscape of prior low-power synthesis methods, analyzes its tradeoffs, reports benchmark results, and discusses its advantages, limitations, applications in IoT and mobile devices, and future improvements such as latency optimization, interconnect binding, support for larger circuits, and AI-based decision making.

II. Background and Problem Context

A. Sources of Power and the Role of Scheduling and Binding

At the behavioral level, power is consumed mainly by computational units (functional units and registers), by communication units, and by peripheral units. Within HLS, poor scheduling leads to unnecessary switching activity and therefore higher power and latency, while a poor binding choice, such as selecting the wrong adder for an operation, increases the switching activity of that unit and wastes power. Allocation also matters: too many units increase area and power, while too few increase latency. The problem addressed here is to assign operations to hardware units so that the least power is wasted, without increasing chip area or causing large latency increases.

B. Prior Low-Power Methods

Several methods have been proposed for low-power scheduling and binding. ILP and LP formulations provide exact solutions but become too complex for large circuits and typically optimize either scheduling or binding separately rather than both together [2]. Force-directed scheduling uses a structural concurrency measure rather than a direct power-cost decision [6]. Simulated annealing and related stochastic methods can handle combined scheduling and binding but suffer from slow, randomized convergence [8]. Latency-oriented schedulers are not power-aware. Against this background, a game theoretic formulation is attractive because it can handle scheduling and binding within a single auction framework while directly minimizing power.

C. Why a Game-Theoretic Formulation

Scheduling and binding both answer the question of which unit performs which task, which is naturally a multi-agent decision problem. Functional units behave like self-interested players that bid according to their power cost, and the auction settles on a stable, low-power outcome. The Nash equilibrium optimizes for all players simultaneously, combining individual and overall cost optimization in a way that simple heuristics do not. This is a *non-cooperative* game: the units decide independently, achieving decentralized optimization, and each player is assumed to be rational and to know the structure of the game [9].

III. Methodology

A. Modeling the Problem as a Game

The key insight is that every hardware unit, such as an adder or a multiplier, acts as a player in a game. Each player bids according to its own power cost, and the player with the lowest cost for an operation is awarded that operation. The stable outcome is found through the Nash equilibrium, at which no player can improve its result by unilaterally changing its decision. The components of the game are the players (functional units), the strategies (which operation a unit performs), the bids (the power cost incurred), the auction type, and the solution concept (the Nash equilibrium).

B. Auction Type Selection

Among the standard auction models, English and Dutch auctions use successively increasing or decreasing bids and are unsuitable here because the power-cost bid is a constant. The second-price sealed-bid auction sets the final value to the second-highest bid, which is not feasible once the bid value has changed. The first-price sealed-bid auction, in which bids are sealed and fixed and the lowest power bid wins, is therefore the best fit for the synthesis problem.

C. Power-Reduction Techniques

Three techniques are embedded within the game theoretic framework to reduce power. In *functional unit sharing*, neighborhood operations that have at least one common input are assigned to the same functional unit, which reduces the number of changing inputs and thus the switching activity. In *path balancing*, the non-synchronous arrival of inputs caused by varying interconnect delays is identified and corrected by adjusting delays at the behavioral level, which avoids redundant computation. In *register assignment*, variables are mapped to registers so that signals arrive on time, reducing glitching without the need for gated registers and additional control logic.

D. Cost Function and Optimal Assignment

The binding cost of assigning a module m to an operation o is expressed as

$$\mathrm{co}(m, o) = P_{\mathrm{sw}} - P_{\mathrm{inp}}, \tag{1}$$

where P_{sw} is the power consumed due to switching and P_{inp} is the power saved due to common (neighborhood) inputs. A lower cost indicates a better unit for that operation. The overall payoff at equilibrium, summed over all N players, is

$$\bar{p} = \sum_{i=1}^{N} p^{i}_{(n_1^*, \ldots, n_N^*)}, \tag{2}$$

where $n_1^*, \ldots, n_N^*$ are the equilibrium strategies of the players. The power-optimal assignment is the one that minimizes the total cost over all feasible module-operation sets,

$$\mathrm{CO}(A^*) = \min_{A \subseteq S} \mathrm{CO}(A), \tag{3}$$

where A is a candidate assignment of operations to units and $\mathrm{CO}(A)$ is its total power cost [1].

IV. Game Theoretic Scheduling

The scheduling process is applied level by level on the DFG. The dataflow graph is read, and the operations that are ready at the start form the top level. The functional units bid for these operations using their power costs, and the Nash equilibrium decides the winners at that level. The winning operations are scheduled, then removed from the graph so that the next level becomes ready, and the process repeats until all operations are scheduled. The output is a complete schedule that specifies which operation runs in which cycle and on which unit, arranged for minimum power. This flow is summarized in Fig. 1.

For binding, the same idea is applied at each control step: the compatible modules for a step are identified, a payoff matrix is built from their power costs, and the Nash equilibrium selects the binding that minimizes total power. The combined scheduling and binding formulation applies the Nash equilibrium only once, using a single unified payoff matrix, and achieves better power reduction than applying scheduling and binding separately in sequence [1].

V. Experimental Results

The benchmark circuits used for evaluation are the differential equation (Diff. eqn), finite impulse response filter (FIR), infinite impulse response filter (IIR), and the larger lattice, elliptic wave filter (EWF), WAVE, and noise-cancel

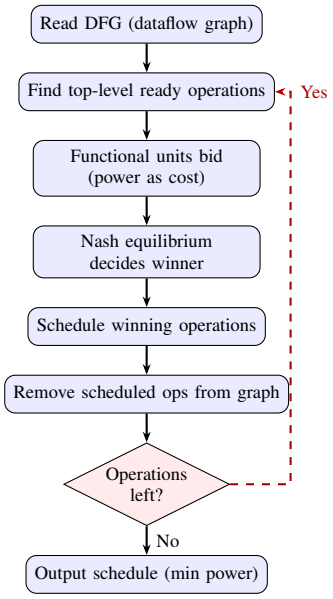

Fig. 1. Flow of the game theoretic scheduling process. Bidding and the Nash equilibrium are applied level by level until all operations are scheduled.

TABLE I
SCHEDULING: POWER (MW) AND % IMPROVEMENT OVER ILP [1]

Circuit	ILP P_{ILP}	GT P_{GT}	Impr.
Diff. eqn	24.8	23.9	3.6%
FIR	82.8	78.7	4.9%
IIR	15.6	14.8	5.1%
Lattice	91.3	82.9	9.2%
EWF	262.8	251.0	4.5%
WAVE	258.2	232.8	9.8%
NC filter	389.7	363.4	6.7%
Average			**6.3%**

filter (NC filter). Power values were obtained by simulation with 100 000 random 16-bit two's complement input vectors using TSMC $0.25\,\mu\mathrm{m}$ cells, and interconnect lengths were estimated using Rent's rule [1]. The results below are drawn from the benchmark tables of Murugavel and Ranganathan and are used here to support the approach.

As shown in Table I, the game theoretic scheduling algorithm yields an average improvement of about 6.3% over the ILP method, with run times comparable to ILP. The binding results in Table II show an average improvement of about 13.9% over the LP-based technique, with the largest single-circuit gain on the IIR filter at 20.6%; this gain is achieved even without interconnect power optimization, because the Nash equilibrium optimizes for all players and because both functional unit and register binding are considered. The combined scheduling and binding results in Table III show an average improvement of about 11.8% over the sequential ILP-plus-LP baseline, with a worst-case latency increase of no more than two cycles. Across all benchmarks there is no increase in area, and run times remain comparable to the baselines.

TABLE II
BINDING: POWER (MW) AND % IMPROVEMENT OVER LP [1]

Circuit	LP P_{LP}	GT P_{GT}	Impr.
Diff. eqn	22.9	20.1	12.2%
FIR	72.0	57.7	19.8%
IIR	15.5	12.3	20.6%
Lattice	71.9	64.1	10.8%
EWF	234.3	210.0	10.4%
WAVE	215.1	196.3	8.7%
NC filter	368.5	312.8	15.1%
Average			**13.9%**

TABLE III
COMBINED SCHEDULING AND BINDING: POWER (MW) AND % IMPROVEMENT OVER ILP [1]

Circuit	ILP P_{ILP}	GT P_{GT}	Impr.
Diff. eqn	22.9	16.0	8.0%
FIR	61.7	50.4	18.3%
IIR	12.8	11.2	12.5%
Lattice	66.3	55.9	15.7%
EWF	204.0	191.3	6.2%
WAVE	201.2	179.2	10.9%
NC filter	324.3	286.7	11.5%
Average			**11.8%**

VI. TRADEOFF ANALYSIS

The approach involves several tradeoffs. In the *power versus latency* tradeoff, power is the primary objective and latency is not co-optimized, so the cost is a small latency increase of at most about two cycles, which is acceptable for power-critical, throughput-tolerant designs. In the *power versus area* tradeoff, no additional modules or multiplexers are introduced, so there is no area overhead, and multiplexer power varies only minimally with the binding solution. In the *quality versus runtime* tradeoff, the Nash equilibrium is NP-complete in general, but restricting the number of players to a few units per type (typically two or three) per game keeps run times comparable to ILP and LP. Finally, in the *scheduling-only versus combined* tradeoff, the combined formulation applies the Nash equilibrium once and gives better power reduction than the separate formulation, at the cost of a more intricate unified payoff matrix.

VII. ADVANTAGES AND LIMITATIONS

The proposed game theoretic technique has several key advantages for power optimization in HLS. Operations are extracted from the DFG and each functional unit is treated as a player. Each participant determines the operation it can perform with the least power expenditure, and the Nash equilibrium then yields the assignment that minimizes the power consumption at the functional unit for each operation. This decreases the switching activity at the functional units and thus minimizes overall power consumption. A further advantage is that this strategy does not increase the hardware area; no new adders, multipliers, or other functional units are added to the design. Instead, the existing hardware components are

used more efficiently through better scheduling and binding, so the hardware area is essentially unchanged and only the assignment of operations to units is optimized.

The approach also has certain limitations. Because it prioritizes minimizing power consumption rather than optimizing latency, additional control cycles may sometimes be required, which can cause a slight increase in delay. In addition, when the number of operations and functional units becomes large, computing the Nash equilibrium becomes more difficult, since a greater number of combinations must be checked, which can make the technique slow [1].

VIII. Applications and Relevance

This strategy is very beneficial for IoT devices, wearables, and mobile devices, because the batteries in these devices are small and the power constraints are highly strict. Lower power means longer battery life, less heat, and better device efficiency. HLS makes hardware design easier, and game theory together with the Nash equilibrium can be used to allocate operations to functional units with lower power consumption, which makes scheduling and binding decisions more systematic. Looking ahead, if artificial intelligence or machine learning is applied, the system can learn from prior design experience. This would allow more intelligent judgments about which hardware unit consumes less power, where latency can be reduced, and how computation time can be minimized for larger circuits.

IX. Future Work

In this work, the main focus is on reducing power consumption. In practical hardware design, however, low power and good speed are equally vital; if only power is reduced, the circuit may become slow. Extending this approach to jointly optimize power and latency is therefore an interesting topic for future exploration. In addition, interconnect binding should be taken into account, because in current chips the power consumption and delay arise not only from functional units but also from wires and interconnects, and the optimization is not complete without considering them. Finally, computing the Nash equilibrium requires much more time for large circuits with many operations and hardware components; to address this, further study can examine faster algorithms, heuristic approaches, search-space reduction methods, or AI and machine learning based approaches that reduce the computation time.

X. Conclusion

This paper has presented a game theoretic approach to the scheduling and binding problem in High-Level Synthesis. By modeling functional units as players in a first-price sealed-bid auction and applying the Nash equilibrium, operations are assigned to suitable functional units in a way that minimizes power consumption without increasing the hardware area. Benchmark data indicate average power improvements of about 6.3% for scheduling, 13.9% for binding, and 11.8% for combined scheduling and binding over conventional baselines, at the cost of only a slight latency increase. The main remaining challenges are the scalability of the Nash equilibrium for large circuits and the absence of an interconnect model. Future work should therefore focus on reducing latency alongside power, incorporating interconnect binding, and making the equilibrium computation faster, so that the approach can be applied effectively to larger and more modern designs.

References

[1] A. K. Murugavel and N. Ranganathan, "A game theoretic approach for power optimization during behavioral synthesis," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 11, no. 6, pp. 1031–1043, Dec. 2003.

[2] W. T. Shiue and C. Chakrabarti, "ILP-based scheme for low-power scheduling and resource binding," in *Proc. Int. Symp. Circuits and Systems*, vol. 3, 2000, pp. 279–282.

[3] C. Small, "Shrinking devices put the squeeze on system packaging," *Electron. Design News*, vol. 39, pp. 41–46, Feb. 1994.

[4] L. Benini and G. De Micheli, "System-level power optimization: Techniques and tools," *ACM Trans. Design Automation of Electron. Syst.*, vol. 5, no. 2, pp. 115–192, Apr. 2000.

[5] D. Gajski, N. Dutt, A. Wu, and Y.-L. Lin, *High-Level Synthesis: Introduction to Chip and System Design*. Norwell, MA: Kluwer, 1992.

[6] P. G. Paulin and J. P. Knight, "Scheduling and binding algorithms for high-level synthesis," in *Proc. Design Automation Conf.*, 1989, pp. 1–6.

[7] S. P. Mohanty, N. Ranganathan, and S. K. Chappidi, "Simultaneous peak and average power minimization during datapath scheduling for DSP processors," in *Proc. Great Lakes Symp. VLSI*, 2003, pp. 215–220.

[8] A. Dasgupta and R. Karri, "Simultaneous scheduling and binding for power minimization during micro-architecture synthesis," in *Proc. Int. Symp. Low-Power Electronics Design*, 1995, pp. 69–74.

[9] J. F. Nash, "Non-cooperative games," *Ann. Math.*, vol. 54, no. 2, pp. 286–295, 1951.

[10] G. Owen, *Game Theory*, 3rd ed. New York: Academic, 1995.

[11] J. W. Friedman, *Game Theory with Applications to Economics*. London, U.K.: Oxford Univ. Press, 1990.

[12] N. Park and A. C. Parker, "Sehwa: A software package for synthesis of pipelines from behavioral specifications," *IEEE Trans. Computer-Aided Design*, vol. 7, pp. 356–370, Mar. 1988.

[13] R. D. McKelvey, A. McLennan, and T. Turocy, *Gambit: Software Tools for Game Theory*, ver. 0.97, 2002.